



Faculty of Technology
University of Delhi

Department of Computer Science & Engineering
CSE-A

Complaint Management System

Software Requirements Specification & Extended Design Report

Subject: **Software Engineering**

Project Team

S.No.	Name	Roll Number
1	Mayank Goyal	24293916114
2	Prateek Binjola	24293916071
3	Deepanshu Sharma	24293916082
4	Krishan Kumar	24293916088
5	Vansh Sahrawat	24293916089

Contents

1	Introduction	4
1.1	Purpose	4
1.2	Scope	4
1.3	Definitions, Acronyms, and Abbreviations	4
1.4	Overview	5
2	Overall Description	6
2.1	Product Perspective	6
2.2	Product Functions	6
2.3	User Classes and Characteristics	7
2.4	Operating Environment	7
2.5	Design and Implementation Constraints	8
2.6	Assumptions and Dependencies	8
3	Specific Requirements	9
3.1	External Interface Requirements	9
3.1.1	User Interfaces	9
3.1.2	Hardware Interfaces	9
3.1.3	Software Interfaces	9
3.1.4	Communication Interfaces	10
3.2	Functional Requirements	10
3.2.1	Triage Inbox	10
3.2.2	Kanban Board	10
3.2.3	Issue Drawer	11
3.2.4	Promote Pipeline	12
3.2.5	Create Complaint	12
3.2.6	Command Palette and Keyboard Shortcuts	13
3.3	Non-Functional Requirements	13
3.4	Use Cases	14
3.4.1	UC-01: Triage a Discord Complaint	14
3.4.2	UC-02: Move Issue on Kanban Board	15
3.4.3	UC-03: Bulk Spam Session	15
3.4.4	UC-04: Create Internal Complaint	16
3.4.5	UC-05: View AI Duplicate Alert	16
4	Software Design	17
4.1	System Architecture	17
4.1.1	Technology Stack Summary	17

4.1.2	Data Flow: Discord Complaint Ingest	18
4.2	Detailed Design	19
4.2.1	Class Diagrams	19
4.2.2	Sequence Diagrams	20
4.3	Database Schema	22
4.3.1	Enum Definitions	23
4.3.2	Prisma Schema (Excerpt)	23
4.3.3	Issue Status Lifecycle	24
4.4	Security Design	25
4.4.1	Authentication and Authorization Mechanisms	25
4.4.2	Security Protocols	25
5	Traceability	27
5.1	Requirement-to-Design Mapping	27
6	Code	30
6.1	Repository	30
6.2	Repository Structure	30
6.3	GitHub Repository & Commit History	31
6.3.1	Repository Overview	31
6.3.2	Contributor Activity Graph	32
6.3.3	Individual Commit Histories	33
7	Deployment	36
7.1	Deployment Architecture	36
7.2	Local Development Setup	36
7.3	Environment Variables	37
7.4	Screenshots of the Working Application	37
8	User Manual	41
8.1	Purpose and Scope	41
8.2	Prerequisites	41
8.3	Dashboard Layout	42
8.4	Triage Inbox	42
8.4.1	Reading the Table	42
8.4.2	Filters	42
8.4.3	Keyboard-Driven Triage	43
8.4.4	Bulk Actions	43
8.5	Active Board	43
8.5.1	Columns	43
8.5.2	Moving Cards	44
8.5.3	Card Information	44
8.5.4	Milestone Progress	44
8.6	Issue Drawer	44
8.6.1	Opening and Closing	44
8.6.2	Drawer Sections	44
8.7	Create Complaint	45
8.8	Command Palette	45
8.9	Troubleshooting	45

8.10 Daily Workflow Recommendation	46
9 Summary	47
9.1 Project Achievements	47
9.2 Key Design Decisions	48
9.3 Lessons Learned	48
9.4 Future Work	48

Chapter 1

Introduction

1.1 Purpose

This document is the Software Requirements Specification (SRS) for the **Complaint Management System** (CMS), a multi-platform open-source bug triage ecosystem. It describes the functional and non-functional requirements, system constraints, external interfaces, and extended design artefacts required to build, deploy, and maintain the system.

The intended audience includes:

- Development team members implementing the system.
- External examiners evaluating the project submission.
- Future maintainers extending the platform.

1.2 Scope

The CMS is designed to solve a real-world pain-point articulated by open-source maintainers: the inability to triage community bug reports efficiently. Three core capabilities distinguish it from existing tools:

1. **Discord Buffer Gateway** — Community complaints arrive via Discord Forum Channels, keeping the upstream GitHub issue tracker pristine.
2. **Proactive AI Deduplication** — A vector-embedding pipeline detects duplicate reports the moment they are submitted.
3. **Decoupled Read-Only Mirror** — A blazing-fast, login-free public frontend lets anyone browse confirmed bugs without an account.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
CMS	Complaint Management System (this project).
SRS	Software Requirements Specification.
Discord Gateway	The Discord Forum Channel used as the community intake buffer for incoming bug reports.
Triage	The act of reviewing, classifying, and routing incoming issues.
Promotion	Moving a validated Discord complaint onto the GitHub issue tracker.
Kanban Board	Visual board tracking active issues across workflow stages.
AI Similarity	Cosine-similarity score between a new report's vector embedding and existing issues in the knowledge base.
ORM	Object-Relational Mapper (Prisma in this project).
SSR	Server-Side Rendering (Next.js App Router).
RSC	React Server Component.
PR	Pull Request on GitHub.
CI	Continuous Integration pipeline status.
CUID	Collision-resistant Unique Identifier used as key.

1.4 Overview

The remainder of this document is structured as follows:

- **Chapter 2** — Overall Description: product perspective, user classes, assumptions, and constraints.
- **Chapter 3** — Specific Requirements: functional requirements, non-functional requirements, and use cases.
- **Chapter 4** — Software Design: architecture, class diagrams, sequence diagrams, database schema, and security design.
- **Chapter 5** — Traceability matrix.
- **Chapter 6** — Code & repository information.
- **Chapter 7** — Deployment screenshots.
- **Chapter 8** — User Manual.
- **Chapter 9** — Summary.

Chapter 2

Overall Description

2.1 Product Perspective

The CMS is a standalone, self-hosted web application that sits between a community of non-technical users (on Discord) and a software development team (on GitHub). Figure 2.1 shows the system context.

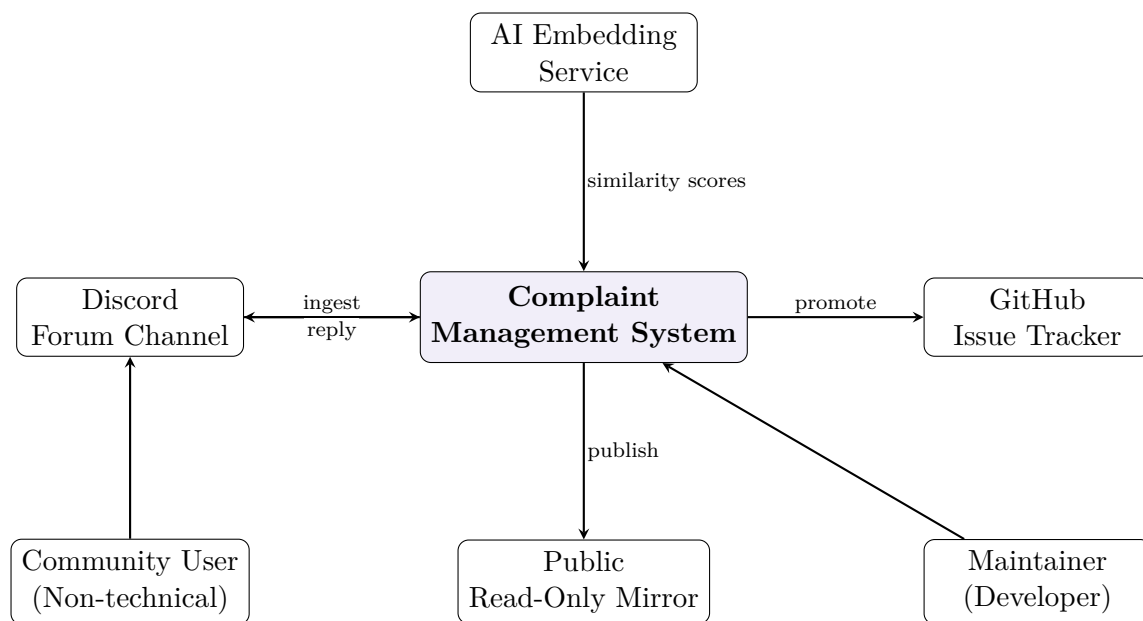


Figure 2.1: System Context Diagram — CMS and its external actors

2.2 Product Functions

At the highest level the system provides:

1. **Triage Inbox** — Displays all Discord-sourced complaints in a dense, sortable table with priority, AI similarity score, reporter avatar, age, and status. Supports keyboard-driven navigation (j/k) and one-key actions (p = promote, x = spam, e = done).
2. **Kanban Board** — Drag-and-drop board for active (GitHub-source) issues across five workflow stages: Backlog, Todo, In Progress, In Review, Done.

3. **Issue Drawer** — Animated slide-over panel with full issue context, dual-tab chat (Internal Notes / Public Reply), assignee and label metadata, linked PRs with CI dots, and context-aware action buttons.
4. **AI Deduplication** — Similarity score stored on each issue; visual progress bar and colour-coded alert in both the triage table and drawer.
5. **Promote Pipeline** — One-click workflow that creates a GitHub issue via REST API, posts a Discord reply with the new issue URL, and moves the complaint into the Backlog column.
6. **Bulk Actions** — Multi-select checkbox + bulk-promote and bulk-spam for high-volume triage sessions.
7. **Command Palette** — `Cmd/Ctrl+K` modal with grouped navigation and action commands.
8. **Create Complaint** — Modal form to directly create internal (GitHub-source) complaints with title, description, and priority.

2.3 User Classes and Characteristics

User Class	Description	Needs
Maintainer	Core developer with full dashboard access	Fast triage, keyboard shortcuts, action buttons, internal notes
Community User	Non-technical end-user on Discord	Zero-friction bug reporting, no account needed
Anonymous Browser	Read-only public mirror visitor	Searchable, SEO-indexed bug list without login

2.4 Operating Environment

- **Server:** Node.js 20 LTS, Next.js 16 (App Router), running on any Linux/macOS host or Docker container.
- **Database:** PostgreSQL 16 (via Docker in development; any managed PostgreSQL service in production).
- **ORM:** Prisma 7 with `@prisma/adapters-pg driver adapter`.
- **Browser:** Modern Chromium/Firefox with JavaScript enabled.
- **External APIs:** Discord Bot API v10, GitHub REST API v2022-11-28.

2.5 Design and Implementation Constraints

- Must use Next.js App Router exclusively (no Pages Router).
- Database schema is managed by Prisma schema file; no raw SQL migrations in application code.
- GitHub and Discord integrations are optional; the system must degrade gracefully when `GITHUB_TOKEN` or `DISCORD_BOT_TOKEN` are not set.
- The public mirror must not require authentication.
- Title word limit is enforced at 200 words.

2.6 Assumptions and Dependencies

- Docker is available on the developer's machine for local PostgreSQL.
- The Discord bot has `Send Messages` and `Read Message History` permissions.
- The GitHub Personal Access Token has `repo` scope.
- AI similarity scores are pre-computed and stored; the live embedding service is a separate concern outside this MVP scope.

Chapter 3

Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

- The web application shall render a responsive, dark-themed dashboard (background #1f2023).
- Navigation shall be accessible via both mouse click and keyboard shortcut without relying solely on either.
- The Triage Inbox shall present data in a scrollable table with sticky header row.
- The Kanban Board shall support drag-and-drop using pointer and keyboard sensors (@dnd-kit).
- The Issue Drawer shall animate in from the right with a 200 ms ease transition.

3.1.2 Hardware Interfaces

No special hardware is required beyond a standard x86-64 or ARM64 server capable of running Node.js 20.

3.1.3 Software Interfaces

Interface	Description
Discord API v10	REST endpoint POST /channels/{id}/messages used to send promotion acknowledgement replies to Discord threads.
GitHub API v2022-11-28	REST endpoint POST /repos/{owner}/{repo}/issues used to create GitHub issues from validated Discord complaints.
PostgreSQL 16	Relational database accessed exclusively through Prisma client; connection via pg driver adapter.

3.1.4 Communication Interfaces

All external API calls use HTTPS/TLS 1.2+. Database connections use TCP within the Docker network or a private cloud VPC.

3.2 Functional Requirements

3.2.1 Triage Inbox

ID	Name	Description
FR-01	List Discord Issues	System shall display all DISCORD-source issues with status <code>TRIAGE_PENDING</code> , <code>TRIAGE_SPAM</code> , or <code>TRIAGE_PROMOTED</code> in the inbox.
FR-02	Filter Triage View	Maintainer can filter by: Pending, All, Flagged Duplicate, Promoted, Spam using tab selectors.
FR-03	Keyboard Navigation	Pressing j/k shall move the focused row cursor down/up; the cursor shall scroll into view.
FR-04	Promote Issue	Pressing p, clicking the promote icon, or using the command palette shall invoke <code>promoteIssue()</code> .
FR-05	Mark Spam	Pressing x or clicking the spam icon shall set status to <code>TRIAGE_SPAM</code> .
FR-06	Mark Done	Pressing e shall set status to <code>DONE</code> .
FR-07	Bulk Select	Selecting 2+ rows shall display the Bulk Action Bar.
FR-08	Bulk Promote	Bulk Action Bar “Promote All” shall call <code>bulkPromote()</code> for each selected issue.
FR-09	Bulk Spam	Bulk Action Bar “Spam All” shall call <code>bulkSpam()</code> in a single <code>updateMany</code> query.
FR-10	Similarity Display	Each row shall display the AI similarity score as a percentage and colour-coded progress bar (green < 30%, amber 30–65%, red > 65%).

3.2.2 Kanban Board

ID	Name	Description
FR-11	Display Board Columns	Board shall render five columns: Backlog, Todo, In Progress, In Review, Done, each showing GitHub-source issues.
FR-12	Drag-and-Drop Cards	Maintainer shall drag an issue card between columns; the UI updates optimistically and the server is called.
FR-13	Card Metadata	Each card shall show: issue number, title, priority dot, label chips, linked PR count with CI status dot, assignee avatars.
FR-14	Milestone Progress	If a milestone exists, the board header shall show a progress bar with closed/total count and percentage.
FR-15	Move Status Persist	<code>moveKanbanCard()</code> shall update <code>Issue.status</code> and <code>lastActivityAt</code> in the database.

3.2.3 Issue Drawer

ID	Name	Description
FR-16	Fetch Issue Detail	Opening the drawer shall call <code>GET /api/issues/{id}</code> and render full issue data.
FR-17	Discord Thread View	For DISCORD-source issues, the drawer shall render the original message and all replies in a thread-style layout.
FR-18	AI Similarity Block	When <code>similarityScore > 0.1</code> , the drawer shall display a colour-coded similarity card with merge/dismiss actions.
FR-19	Dual Chat Tabs	Drawer shall provide “Internal Notes” (private) and “Public Reply” (syncs to Discord) tabs.
FR-20	Focus Mode	Pressing <code>f</code> or the expand icon shall toggle the drawer to full-width (90vw).
FR-21	Promote from Drawer	Footer “Promote to GitHub” button shall invoke <code>promoteIssue()</code> and close the drawer.

ID	Name	Description
FR-22	Copy Link	Clicking the copy icon shall write the current URL to clipboard via <code>navigator.clipboard</code> .
FR-23	Metadata Sidebar	Right panel shall show assignees, labels, milestone, GitHub issue number, and linked PRs with CI status.

3.2.4 Promote Pipeline

ID	Name	Description
FR-24	Create GitHub Issue	When GitHub integration is enabled, <code>promoteIssue()</code> shall call GitHub REST API to create a new issue with title, formatted body, and labels.
FR-25	Discord Notification	When Discord integration is enabled and the issue has a <code>discordThreadId</code> , the system shall post a promotion comment back to the Discord thread.
FR-26	Label Retry	If GitHub returns 422 (labels not found), the system shall retry without labels.
FR-27	Status Update	Promoted issue shall have <code>source</code> set to <code>GITHUB</code> and <code>status</code> set to <code>BACKLOG</code> .

3.2.5 Create Complaint

ID	Name	Description
FR-28	Open Create Modal	Pressing <code>c</code> or clicking “New Complaint” in the top bar shall open the create modal.
FR-29	Validate Title	Title must not exceed 200 words; violation shall throw a descriptive error.
FR-30	Persist Complaint	Submitting the form shall invoke <code>createIssue()</code> which creates an issue with <code>source=GITHUB</code> , <code>status=TODO</code> , and auto-increments the per-organisation issue number.

ID	Name	Description
FR-31	Post-Create Navigation	After successful creation, the app shall navigate to the Active Board.

3.2.6 Command Palette and Keyboard Shortcuts

ID	Name	Description
FR-32	Open Palette	<code>Ctrl/Cmd+K</code> shall open the command palette powered by <code>cmdk</code> .
FR-33	Navigate Commands	Palette groups: Navigate (triage, board, settings), Actions (promote, spam, done, merge, flag), View (focus mode).
FR-34	Go-to Sequences	<code>g→t</code> (triage), <code>g→b</code> (board), <code>g→s</code> (settings) shall navigate via <code>tinykeys</code> two-key sequences.

3.3 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Triage page initial server response shall complete within 500 ms on a local machine with ≤ 1000 issues.
NFR-02	Security	All maintainer actions (promote, spam, create) shall execute as Next.js Server Actions; no sensitive logic runs client-side.
NFR-03	Security	API tokens (<code>GITHUB_TOKEN</code> , <code>DISCORD_BOT_TOKEN</code>) shall never be exposed to the browser; they shall only be read from environment variables on the server.
NFR-04	Reliability	Drag-and-drop moves shall employ optimistic UI with automatic rollback on server failure.
NFR-05	Usability	All primary triage actions shall be completable without lifting hands from the keyboard.
NFR-06	Scalability	Database queries for the board and triage shall include only necessary joins via Prisma <code>include</code> , avoiding N+1 queries.

ID	Category	Requirement
NFR-07	Maintainability	TypeScript strict mode shall be enabled; all server action function signatures shall be explicitly typed.
NFR-08	Accessibility	Interactive elements shall have <code>aria-label</code> attributes; keyboard focus shall be visible.
NFR-09	Deployability	A single <code>npm run setup</code> command shall bootstrap the full local development environment.
NFR-10	Data Integrity	<code>IssueAssignee</code> and <code>IssueLabel</code> join tables shall cascade-delete on parent <code>Issue</code> deletion.

3.4 Use Cases

3.4.1 UC-01: Triage a Discord Complaint

UC-01 — Triage Discord Complaint

Actors: Maintainer

Preconditions: Maintainer is on `/dashboard/triage`; at least one DISCORD-source complaint with status `TRIAGE_PENDING` exists.

Main Flow:

1. Maintainer presses `j` to move cursor to first pending complaint.
2. System highlights the row and scrolls it into view.
3. Maintainer presses `Enter` to open the Issue Drawer.
4. Drawer slides in; system fetches full issue data from `/api/issues/{id}`.
5. Maintainer reviews the Discord thread context and AI similarity score.
6. Maintainer presses `p` (or clicks “Promote to GitHub” in drawer).
7. System calls `promoteIssue()`: creates GitHub issue, posts Discord reply, updates DB.
8. Toast notification confirms promotion; drawer closes; triage list refreshes.

Alternate Flow A (Spam): At step 6, maintainer presses `x`; issue is marked `TRIAGE_SPAM`.

Postconditions: Issue has `source=GITHUB`, `status=BACKLOG`, appears on Kanban board.

3.4.2 UC-02: Move Issue on Kanban Board

UC-02 — Move Kanban Card

Actors: Maintainer

Preconditions: At least one issue exists in the BACKLOG or TODO column.

Main Flow:

1. Maintainer drags an issue card from **Todo** to **In Progress**.
2. UI optimistically moves the card.
3. System calls `moveKanbanCard(issueId, ``IN_PROGRESS'')`.
4. Database updates `Issue.status` and `lastActivityAt`.
5. Toast confirms move.

Alternate Flow A (Failure): Server action throws; UI rolls back card to original column; error toast shown.

Postconditions: Issue status is `IN_PROGRESS` in database.

3.4.3 UC-03: Bulk Spam Session

UC-03 — Bulk Spam

Actors: Maintainer

Main Flow:

1. Maintainer selects multiple rows using checkboxes.
2. Bulk Action Bar appears at the bottom of the screen.
3. Maintainer clicks “Spam All”.
4. System calls `bulkSpam(selectedIds)`.
5. All selected issues updated to `TRIAGE_SPAM` in a single `updateMany`.
6. Selection clears; triage view refreshes.

Postconditions: All selected issues have `status=TRIAGE_SPAM`.

3.4.4 UC-04: Create Internal Complaint

UC-04 — Create Complaint

Actors: Maintainer

Main Flow:

1. Maintainer presses `c` to open the Create Complaint modal.
2. Fills in title, description, and priority.
3. Presses `Ctrl+Enter` or clicks “Create Complaint”.
4. System validates title ≤ 200 words.
5. `createIssue()` creates DB record with `source=GITHUB`, `status=TODO`.
6. App navigates to Active Board; new card appears in Todo column.

Postconditions: New issue persisted with auto-incremented per-org number.

3.4.5 UC-05: View AI Duplicate Alert

UC-05 — AI Duplicate Detection

Actors: Maintainer (triggered automatically on issue ingest)

Main Flow:

1. Maintainer opens the Issue Drawer for a complaint with `similarityScore > 0.1`.
2. Drawer renders the AI Similarity section with progress bar and percentage badge.
3. If `duplicateOfId` is set, the UI shows “Possible duplicate of #N” with Merge and Dismiss buttons.
4. Maintainer clicks “Merge into #N” (phase-2 placeholder in current build; toast shown).

Postconditions: (Phase 2) Issue marked `CLOSED` with `duplicateOfId` set.

Chapter 4

Software Design

4.1 System Architecture

The CMS follows a **Server-Centric Layered Architecture** enabled by the Next.js 16 App Router. Figure 4.1 illustrates the four principal layers.

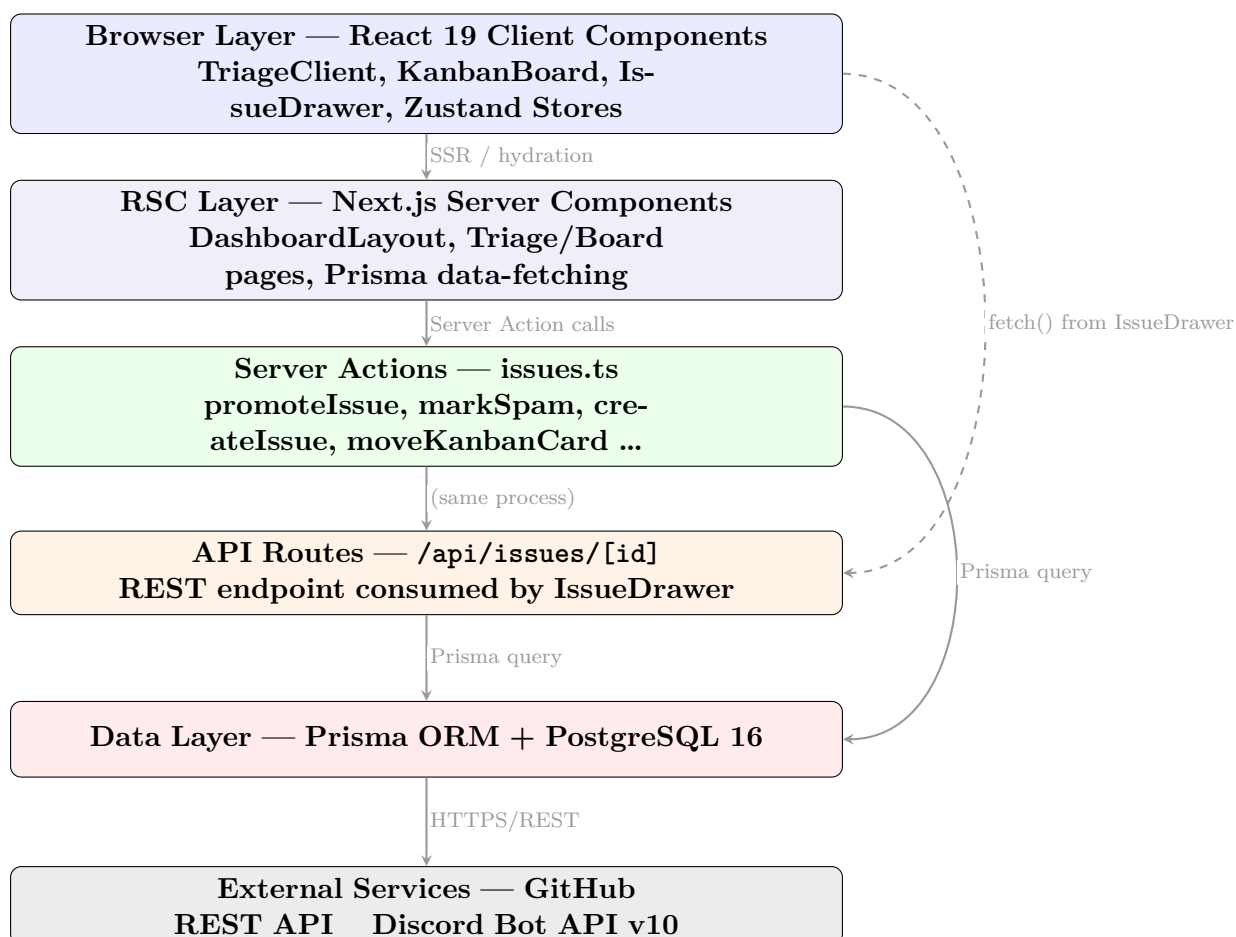


Figure 4.1: CMS Layered System Architecture

4.1.1 Technology Stack Summary

Technology	Role
Next.js 16 (App Router)	Full-stack React framework; RSCs for data fetch, Server Actions for mutations.
React 19	UI component library.
TypeScript 5	Static typing across the entire codebase.
Prisma 7	Type-safe ORM; schema migrations via <code>prisma db push</code> .
PostgreSQL 16	Primary relational database.
Zustand 5	Lightweight client-side state management (drawer, bulk-select, triage cursor).
@dnd-kit	Accessible drag-and-drop for Kanban board.
Framer Motion 12	Drawer animation (<code>AnimatePresence</code> , spring transitions).
TanStack Query 5	Optional server-state caching layer.
Tiptap 3	Rich-text editor for note/reply composer.
cmdk	Headless command palette.
tinykeys 3	Keyboard shortcut binding (including two-key sequences).
Tailwind CSS 4	Utility-first styling.
Sonner 2	Toast notification system.
lucide-react	Icon set.

4.1.2 Data Flow: Discord Complaint Ingest

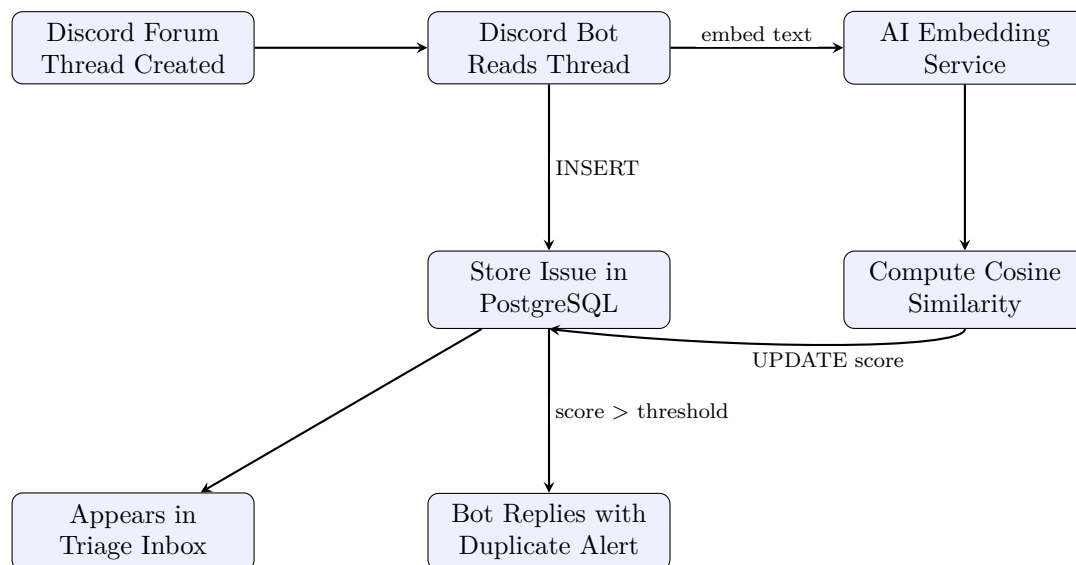


Figure 4.2: Discord Complaint Ingest and AI Deduplication Data Flow

4.2 Detailed Design

4.2.1 Class Diagrams

Domain Model — Core Entities

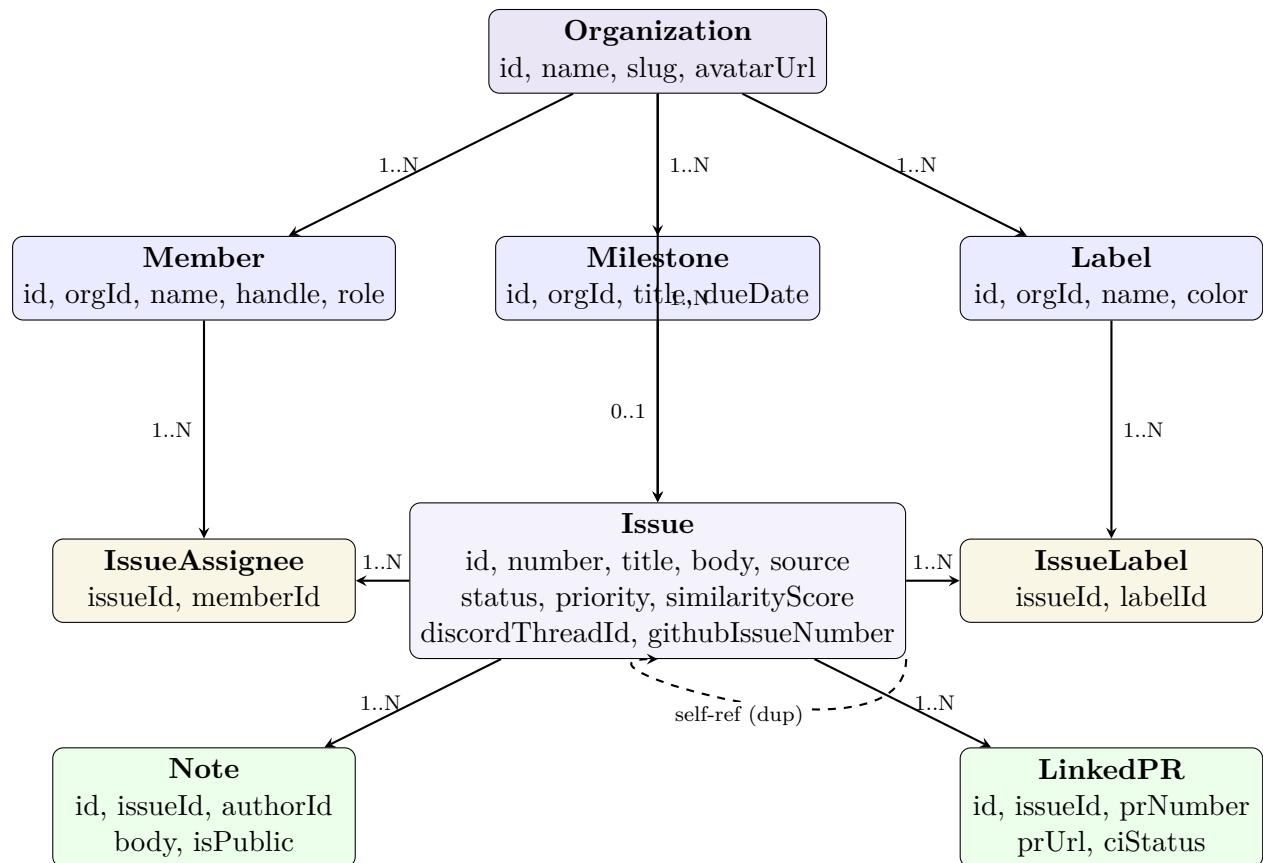


Figure 4.3: Domain Model Class Diagram (D-CL-01)

Component Architecture Class Diagram

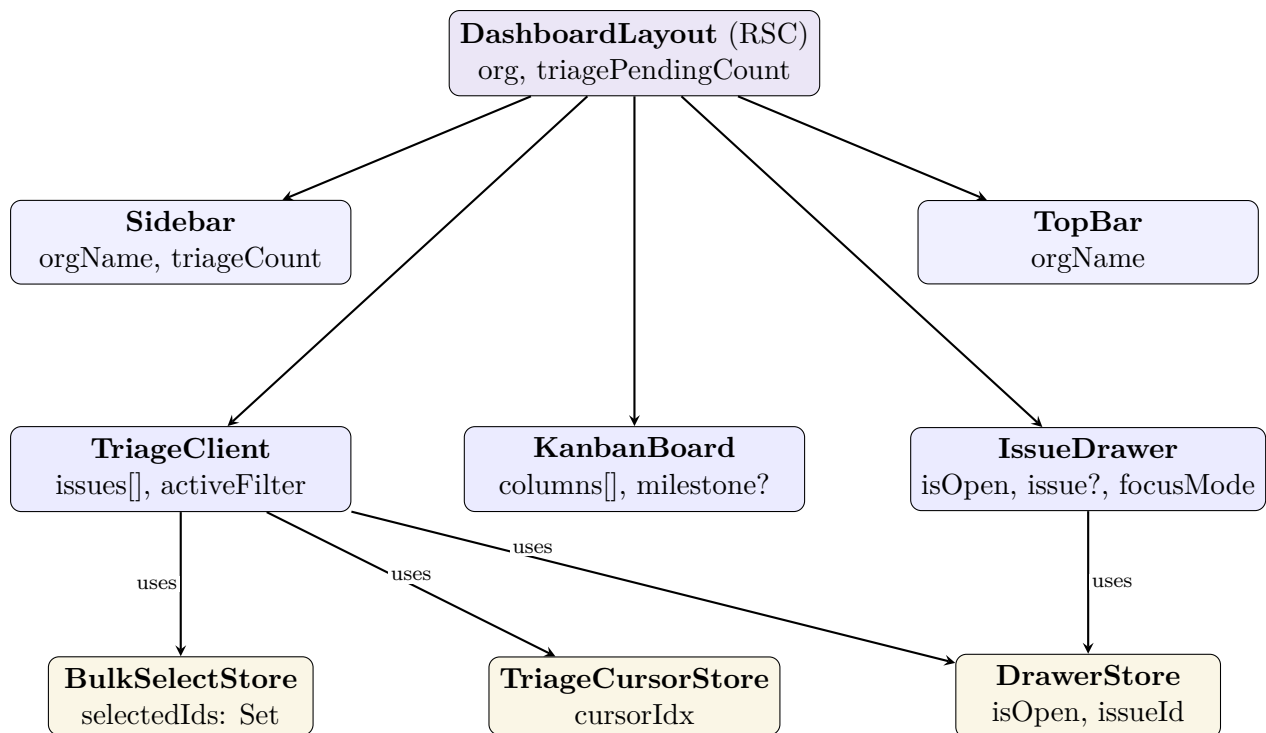


Figure 4.4: Component Architecture Class Diagram (D-CL-02)

4.2.2 Sequence Diagrams

SD-01: Promote Discord Issue to GitHub

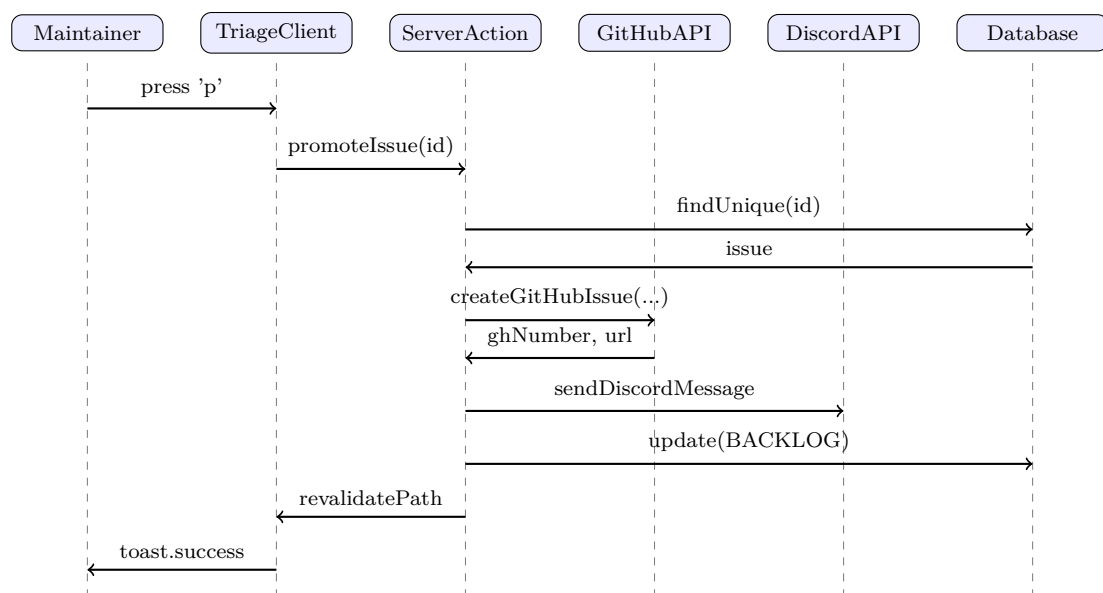


Figure 4.5: Sequence Diagram: Promote Issue (D-SQ-01)

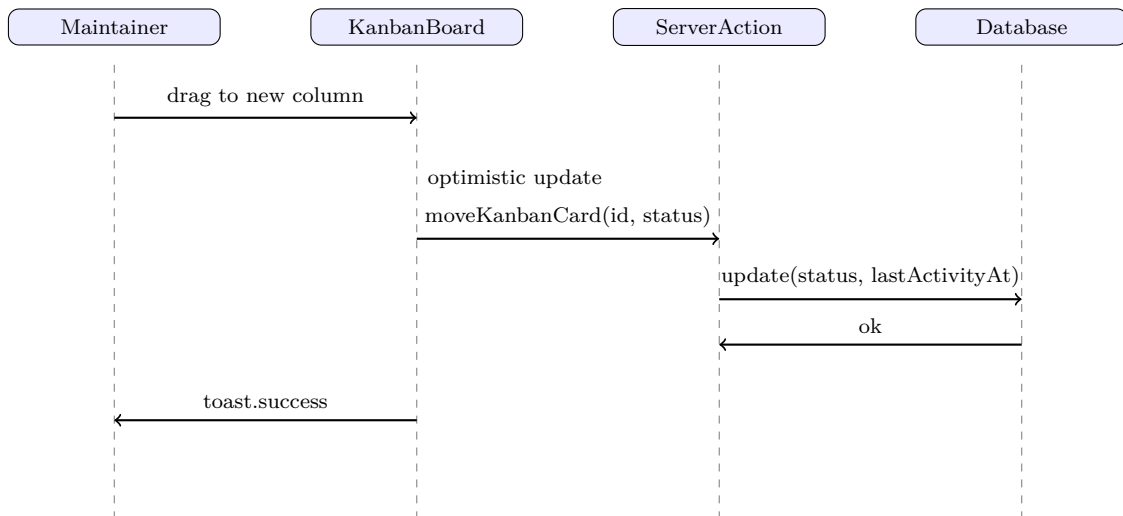
SD-02: Kanban Drag-and-Drop

Figure 4.6: Sequence Diagram: Kanban Move (D-SQ-02)

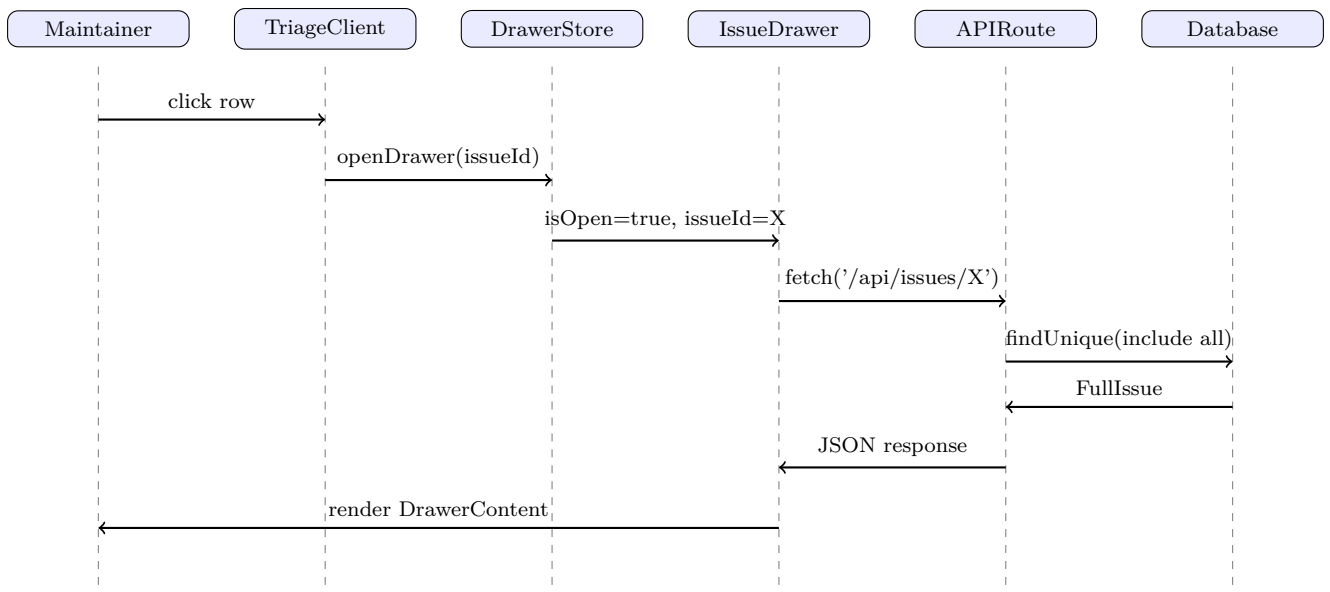
SD-03: Open Issue Drawer

Figure 4.7: Sequence Diagram: Open Issue Drawer (D-SQ-03)

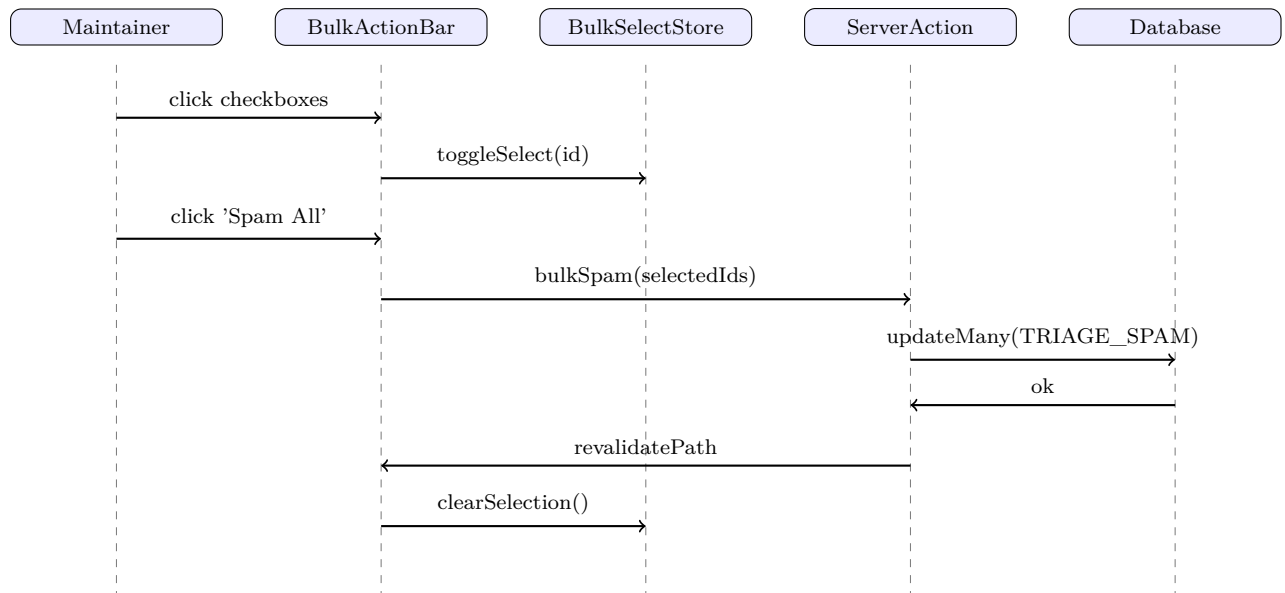
SD-04: Bulk Spam

Figure 4.8: Sequence Diagram: Bulk Spam (D-SQ-04)

4.3 Database Schema

Figure 4.9 presents the complete Entity-Relationship Diagram for the CMS PostgreSQL database managed by Prisma.

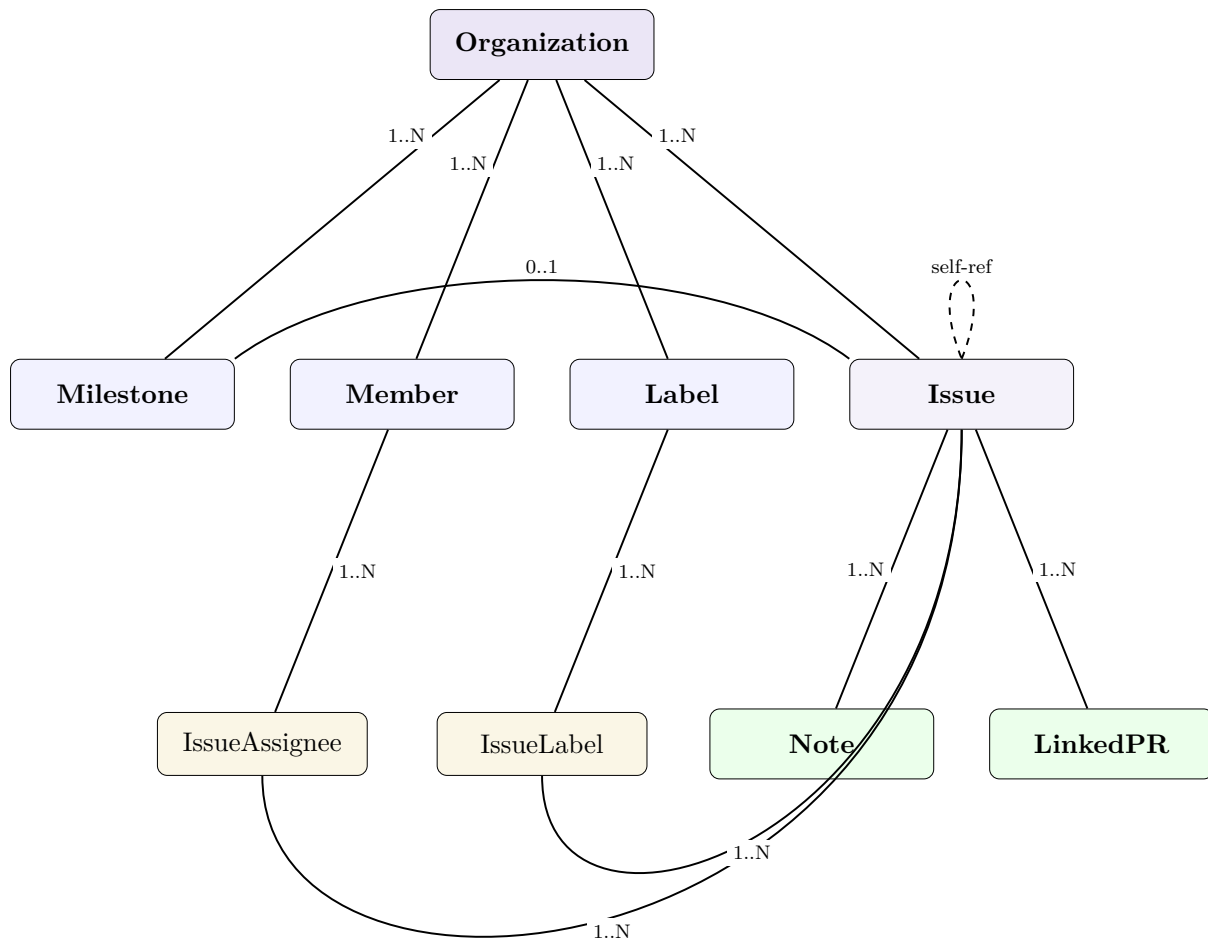


Figure 4.9: Entity-Relationship Diagram (D-DB-01)

4.3.1 Enum Definitions

Enum	Values
IssueSource	DISCORD GITHUB
IssueStatus	TRIAGE_PENDING TRIAGE_SPAM TRIAGE_PROMOTED BACKLOG TODO IN_PROGRESS IN_REVIEW DONE CLOSED
Priority	NONE LOW MEDIUM HIGH CRITICAL

4.3.2 Prisma Schema (Excerpt)

Listing 4.1: Core Issue model from schema.prisma

```

1 model Issue {
2   id          String      @id @default(cuid())
3   number      Int
4   title       String

```

```

5  body          String          @db.Text
6  source        IssueSource
7  status        IssueStatus @default(TRIAGE_PENDING)
8  priority      Priority        @default(NONE)
9  orgId         String
10 org           Organization    @relation(...)
11
12 // Discord metadata
13 discordThreadId String?
14 discordAuthorName String?
15 discordReplies  Json?
16
17 // GitHub (post-promotion)
18 githubIssueNumber Int?
19 githubIssueUrl   String?
20 linkedPRs       LinkedPR []
21
22 // AI deduplication
23 similarityScore  Float?
24 duplicateOfId   String?
25 duplicateOf     Issue? @relation("Duplicates", ...)
26 duplicates      Issue[] @relation("Duplicates")
27
28 notes          Note[]
29 createdAt      DateTime @default(now())
30 updatedAt      DateTime @updatedAt
31 lastActivityAt DateTime @default(now())
32
33 @@unique([orgId, number])
34 }

```

4.3.3 Issue Status Lifecycle

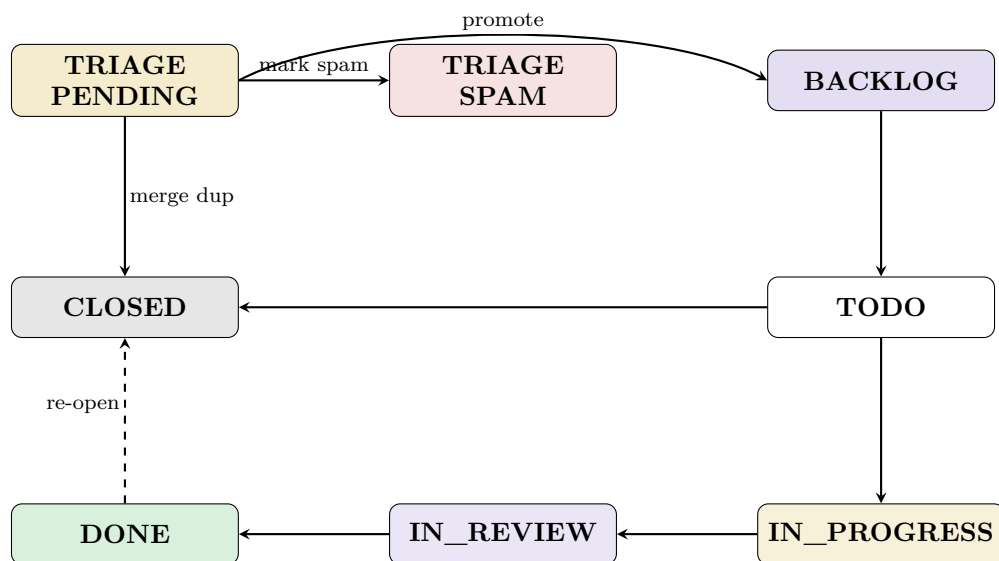


Figure 4.10: Issue Status State Machine

4.4 Security Design

4.4.1 Authentication and Authorization Mechanisms

The current MVP employs a **mock session** strategy suitable for a local / private-network deployment. The `lib/mockSession.ts` module returns a hardcoded maintainer identity. In the planned production phase the following mechanisms will be integrated:

1. **NextAuth.js** with GitHub OAuth provider — maintainers log in with their GitHub account.
2. **Organisation Membership Verification** — after OAuth, the system checks that the authenticated user is a member of the configured GitHub organisation via the GitHub API before granting dashboard access.
3. **Role-Based Access** — the `Member.role` field (currently always "maintainer") will gate actions:
 - **Viewer** — read-only triage and board access.
 - **Maintainer** — promote, spam, label, assign.
 - **Admin** — settings, label creation, member management.
4. **Public Mirror** — the read-only public interface requires no authentication; it queries a dedicated read replica with no write permissions.

4.4.2 Security Protocols

Control (ID)	Description
D-SEC-01	Server-Only Secrets — <code>GITHUB_TOKEN</code> and <code>DISCORD_BOT_TOKEN</code> are read exclusively inside Server Actions and API Route handlers. They are never imported by client components. Next.js statically analyses imports and will throw a build error if a server-only module is imported in a client component.
D-SEC-02	Server Actions as Secure Endpoints — All mutation operations (promote, spam, create, move) are implemented as "use server" functions. The runtime enforces that these are called only from trusted server contexts.
D-SEC-03	Input Validation — The <code>validateTitleWordLimit()</code> function enforces a 200-word cap on all titles before any database write.
D-SEC-04	SQL Injection Prevention — All queries pass through Prisma's parameterised query builder. No raw SQL is used in application code.

Control (ID)	Description
D-SEC-05	HTTPS/TLS — All external API calls (GitHub, Discord) use <code>https://</code> endpoints. Production deployment must terminate TLS at the reverse proxy or CDN layer.
D-SEC-06	Cascade Delete Protection — Foreign-key cascade rules in the Prisma schema ensure that deleting an <code>Issue</code> automatically removes its <code>Note</code> , <code>IssueAssignee</code> , <code>IssueLabel</code> , and <code>LinkedPR</code> records, preventing orphan data accumulation.
D-SEC-07	No PII in URLs — The Issue Drawer fetches issue detail via <code>/api/issues/{id}</code> where <code>id</code> is an opaque CUID, not a user handle or email address.
D-SEC-08	Content Security — Discord <code>discordReplies</code> JSON is stored verbatim and rendered as plain text (<code>whitespace-pre-wrap</code>), not as <code>dangerouslySetInnerHTML</code> , preventing XSS injection from untrusted Discord content.
D-SEC-09	Grace-Degrade on Missing Credentials — Both <code>isGitHubIntegrationEnabled()</code> and <code>isDiscordBotEnabled()</code> check for the presence of env vars before attempting external API calls, preventing unhandled exceptions when credentials are absent.

Chapter 5

Traceability

5.1 Requirement-to-Design Mapping

The table below maps each functional requirement (FR), non-functional requirement (NFR), and use case (UC) to the design artefacts that realise it.

Req./UC ID	Name	Design ID(s)	Design Artefact
UC-01	Triage Discord Complaint	D-CL-01, D-CL-02, D-SQ-01	Class Diagram, Sequence Diagram
UC-02	Move Kanban Card	D-CL-02, D-SQ-02	Class Diagram, Sequence Diagram
UC-03	Bulk Spam Session	D-CL-02, D-SQ-04	Class Diagram, Sequence Diagram
UC-04	Create Internal Complaint	D-CL-01, D-CL-02	Class Diagram
UC-05	AI Duplicate Alert	D-CL-01, D-DB-01	Class Diagram, ERD
FR-01	List Discord Issues	D-CL-02	Component Class Diagram
FR-02	Filter Triage View	D-CL-02	Component Class Diagram
FR-03	Keyboard Navigation	D-CL-02	Component Class Diagram
FR-04	Promote Issue	D-CL-01, D-SQ-01	Class Diagram, Sequence Diagram
FR-05	Mark Spam	D-CL-01, D-SQ-04	Class Diagram, Sequence Diagram
FR-06	Mark Done	D-CL-01	Class Diagram
FR-07	Bulk Select	D-CL-02	Component Class Diagram
FR-08	Bulk Promote	D-CL-02, D-SQ-04	Class Diagram, Sequence Diagram
FR-09	Bulk Spam	D-CL-02, D-SQ-04	Class Diagram, Sequence Diagram

Req./UC ID	Name	Design ID(s)	Design Artefact
FR-10	Similarity Display	D-CL-01, D-DB-01	Class Diagram, ERD
FR-11	Display Board Columns	D-CL-02	Component Class Diagram
FR-12	Drag-and-Drop Cards	D-CL-02, D-SQ-02	Class Diagram, Sequence Diagram
FR-13	Card Metadata	D-CL-01, D-CL-02	Class Diagram
FR-14	Milestone Progress	D-CL-01, D-DB-01	Class Diagram, ERD
FR-15	Move Status Persist	D-CL-01, D-SQ-02	Class Diagram, Sequence Diagram
FR-16	Fetch Issue Detail	D-CL-02, D-SQ-03	Class Diagram, Sequence Diagram
FR-17	Discord Thread View	D-CL-01, D-CL-02	Class Diagram
FR-18	AI Similarity Block	D-CL-01, D-DB-01	Class Diagram, ERD
FR-19	Dual Chat Tabs	D-CL-02	Component Class Diagram
FR-20	Focus Mode	D-CL-02	Component Class Diagram
FR-21	Promote from Drawer	D-CL-02, D-SQ-01	Class Diagram, Sequence Diagram
FR-22	Copy Link	D-CL-02	Component Class Diagram
FR-23	Metadata Sidebar	D-CL-01, D-CL-02	Class Diagram
FR-24	Create GitHub Issue	D-CL-01, D-SQ-01	Class Diagram, Sequence Diagram
FR-25	Discord Notification	D-CL-01, D-SQ-01	Class Diagram, Sequence Diagram
FR-26	Label Retry	D-CL-01	Class Diagram
FR-27	Status Update	D-CL-01, D-DB-01	Class Diagram, ERD
FR-28	Open Create Modal	D-CL-02	Component Class Diagram
FR-29	Validate Title	D-CL-01	Class Diagram
FR-30	Persist Complaint	D-CL-01, D-DB-01	Class Diagram, ERD
FR-31	Post-Create Navigation	D-CL-02	Component Class Diagram
FR-32	Open Palette	D-CL-02	Component Class Diagram
FR-33	Navigate Commands	D-CL-02	Component Class Diagram
FR-34	Go-to Sequences	D-CL-02	Component Class Diagram

Req./UC ID	Name	Design ID(s)	Design Artefact
NFR-01	Performance	D-CL-02	Component Class Diagram (RSC data fetch)
NFR-02	Security (Server Actions)	D-SEC-02	Security List
NFR-03	Security (Secrets)	D-SEC-01	Security List
NFR-04	Reliability (Optimistic UI)	D-SQ-02	Sequence Diagram
NFR-05	Keyboard Usability	D-CL-02	Component Class Diagram
NFR-06	Scalability (N+1)	D-DB-01	ERD / Prisma includes
NFR-07	TypeScript Strict	D-CL-01, D-CL-02	Class Diagrams
NFR-08	Accessibility	D-CL-02	Component Class Diagram
NFR-09	Deployability	—	Deployment Section
NFR-10	Data Integrity (Cascade)	D-DB-01, D-SEC-06	ERD, Security List

Chapter 6

Code

6.1 Repository

The complete source code for the Complaint Management System is hosted on GitHub. The repository contains the full Next.js application, Prisma schema, Docker scripts, and documentation.

Repository URL:

<https://github.com/Unique-45/Complaint-Management-System>

Default Branch: main **License:** MIT

6.2 Repository Structure

Listing 6.1: Project directory tree (key files)

```
1  complaint-management-system/
2  |-- src/
3  |   |-- app/
4  |   |   |-- api/issues/[id]/route.ts      # REST endpoint for drawer
5  |   |   |-- dashboard/
6  |   |   |   |-- layout.tsx                # DashboardLayout RSC
7  |   |   |   |-- triage/page.tsx           # Triage Inbox page
8  |   |   |   |-- board/page.tsx           # Kanban Board page
9  |   |   |   `-- settings/page.tsx
10 |   |-- components/
11 |   |   |-- triage/TriageClient.tsx        # Triage table (client)
12 |   |   |-- board/KanbanBoard.tsx         # DnD Kanban (client)
13 |   |   |-- drawer/IssueDrawer.tsx        # Slide-over drawer (client)
14 |   |   |-- layout/
15 |   |   |   KeyboardHandler
16 |   |   |   |-- issue/CreateIssueModal.tsx
17 |   |   |-- server/actions/issues.ts      # All server actions
18 |   |   |-- lib/
19 |   |   |   |-- prisma.ts                 # Prisma client singleton
20 |   |   |   |-- utils.ts                 # Shared helpers & metadata
21 |   |   |   |-- integrations/
22 |   |   |   |   |-- github.ts             # GitHub API client
23 |   |   |   |   |-- discord.ts           # Discord API client
```

```

23 |   |-- stores/                                # Zustand client stores
24 |       |-- drawerStore.ts
25 |       |-- bulkSelectStore.ts
26 |       |-- triageCursorStore.ts
27 |-- prisma/
28 |   |-- schema.prisma                        # Database schema
29 |   |-- seed.ts                             # Demo data seeder
30 |-- scripts/                               # db-up, db-reset, setup-local
31 |-- docker-compose.yml
32 |-- next.config.ts
33 |-- package.json

```

6.3 GitHub Repository & Commit History

The project is hosted publicly at:

https://github.com/Unique-45/Complaint_Management_System

6.3.1 Repository Overview

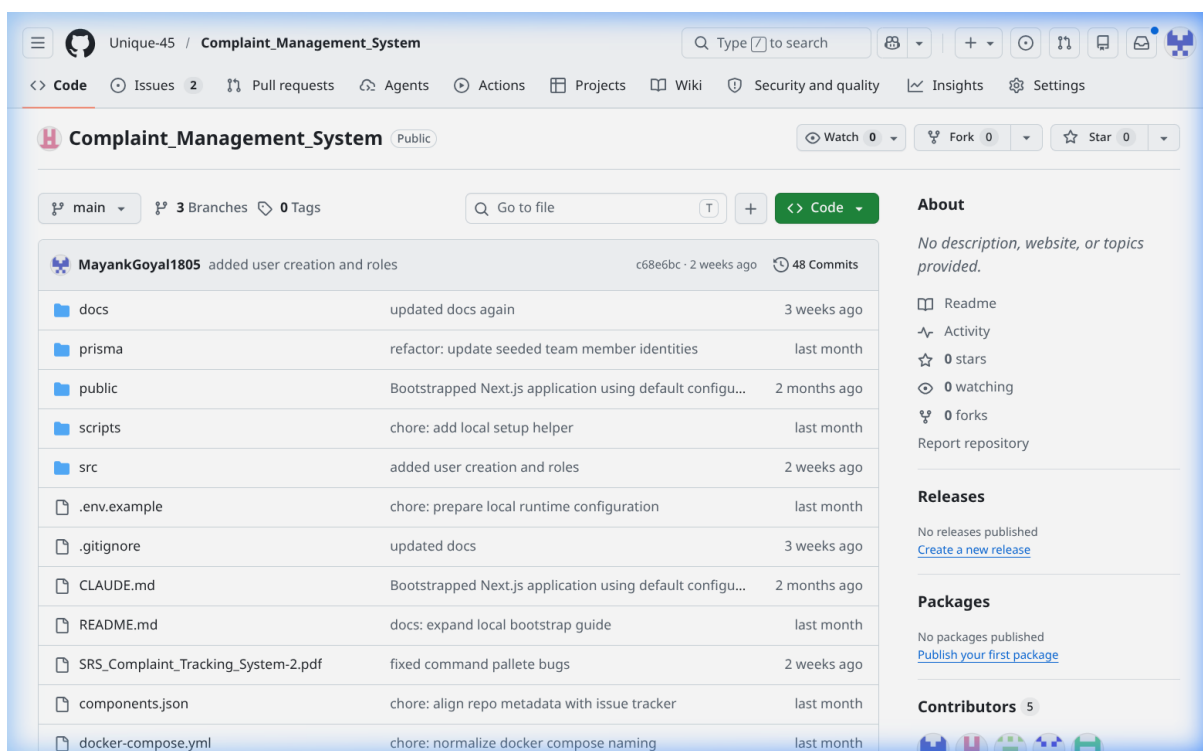


Figure 6.1: GitHub Repository — project structure and README on the main branch

6.3.2 Contributor Activity Graph

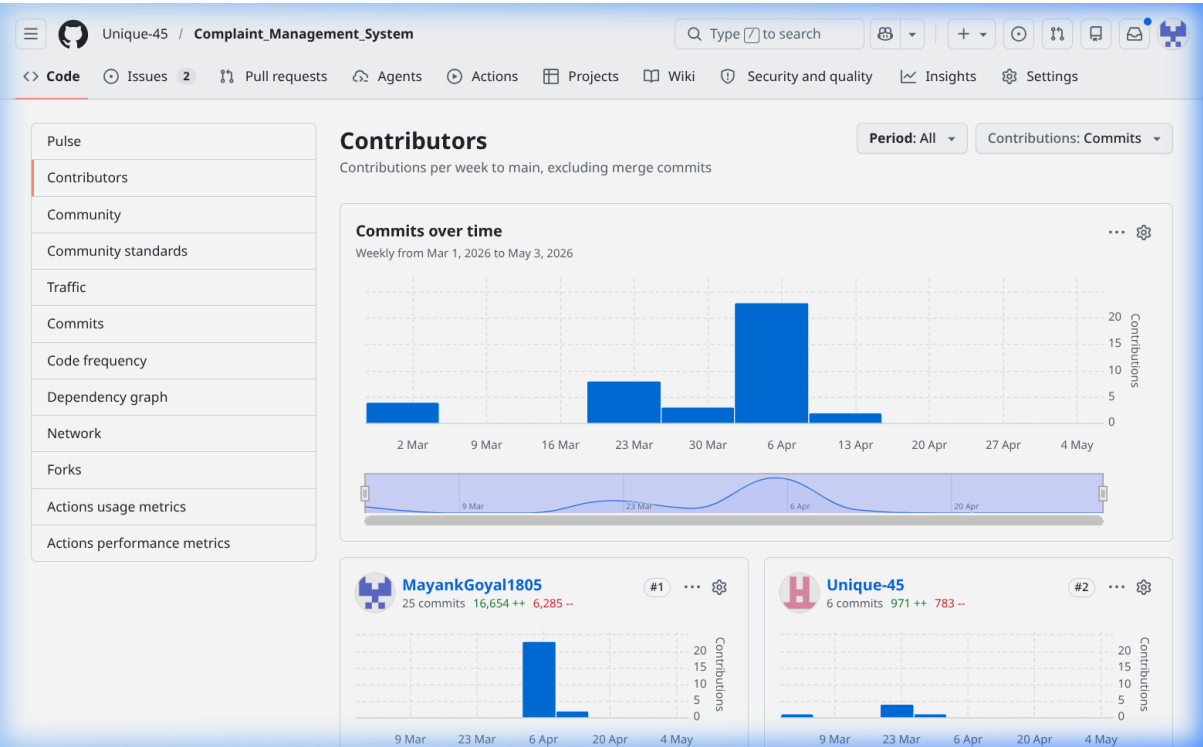


Figure 6.2: GitHub Insights — contributor activity across the full project timeline

6.3.3 Individual Commit Histories

Mayank Goyal (mayankgoyal1805)

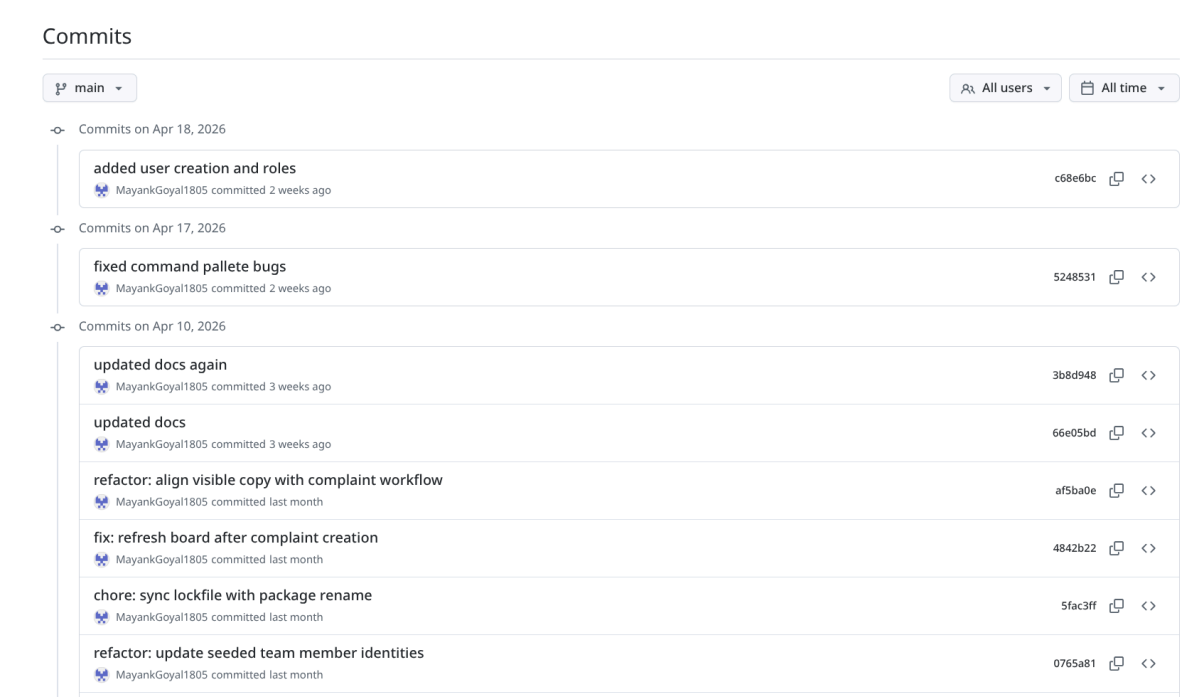


Figure 6.3: Mayank Goyal commits — page 1

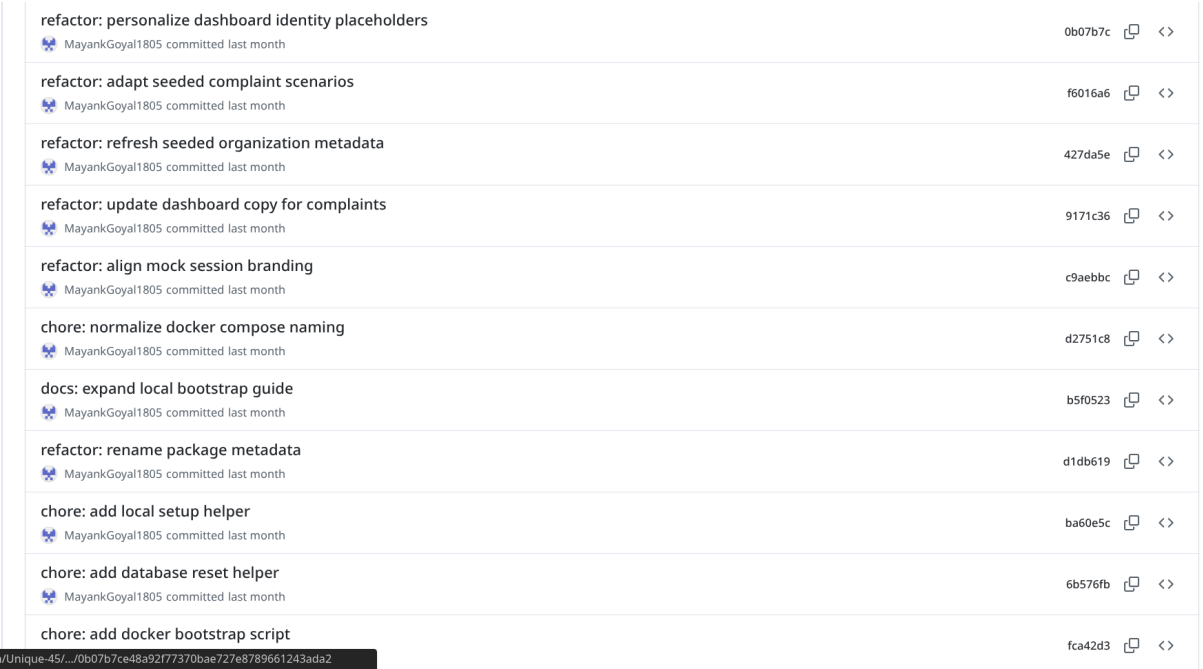


Figure 6.4: Mayank Goyal commits — page 2

docs: add local setup and run instructions MayankGoyal1805 committed last month	d9494c9		
chore: prepare local runtime configuration MayankGoyal1805 committed last month	38900d1		
chore: update prisma and local database setup MayankGoyal1805 committed last month	73d43d8		
feat: add issue management workflows and ui MayankGoyal1805 committed last month	8685c38		
refactor: replace legacy app shell with issue tracker layout MayankGoyal1805 committed last month	34e7070		
chore: align repo metadata with issue tracker MayankGoyal1805 committed last month	a7be54e		

Figure 6.5: Mayank Goyal commits — page 3

Vansh Sahrawat (Unique-45)

Commits on Mar 30, 2026	fix: Resolve package-lock.json merge conflicts Unique-45 committed on Mar 30	a37ca12		
Commits on Mar 27, 2026	Resolve merge conflicts - accept feature branch Unique-45 committed on Mar 27	597786f		
	Configure Prisma seed and update gitignore for generated files Unique-45 committed on Mar 27	8d770a7		
	feat(db): Create seed script with test users and issues Unique-45 committed on Mar 27	8bb3d32		
Commits on Mar 26, 2026	feat(db): Create and apply initial database migrations Unique-45 committed on Mar 26	041161a		
	feat(db): Initialize Prisma schema with User, Issue, Comment models Unique-45 committed on Mar 26	fa5e4d7		
Commits on Mar 1, 2026	Format team members list in README.md Unique-45 authored on Mar 1	Verified 5902aa4		

Figure 6.6: Vansh Sahrawat commits (Unique-45)

Deepanshu Sharma (deepanshu-shar)

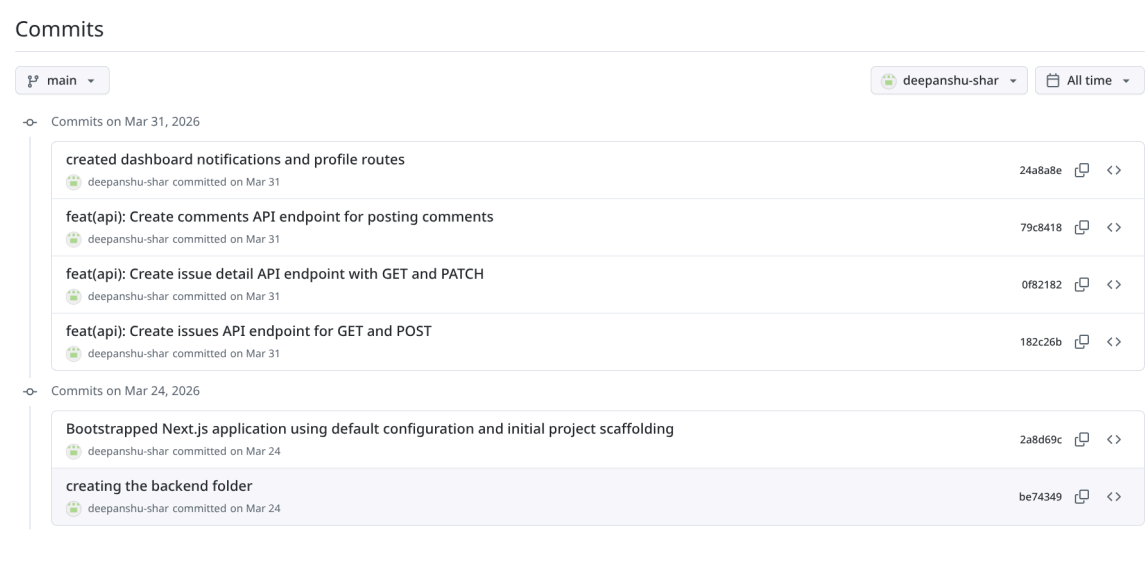


Figure 6.7: Deepanshu Sharma commits (deepanshu-shar)

Krishan Kumar (krishan1707)

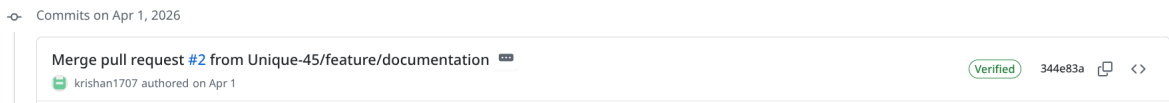


Figure 6.8: Krishan Kumar commits (krishan1707)

Prateek Binjola (prateekbinjola)

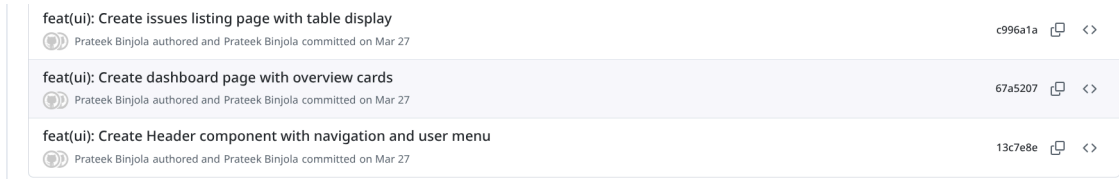


Figure 6.9: Prateek Binjola commits (prateekbinjola)

Chapter 7

Deployment

7.1 Deployment Architecture

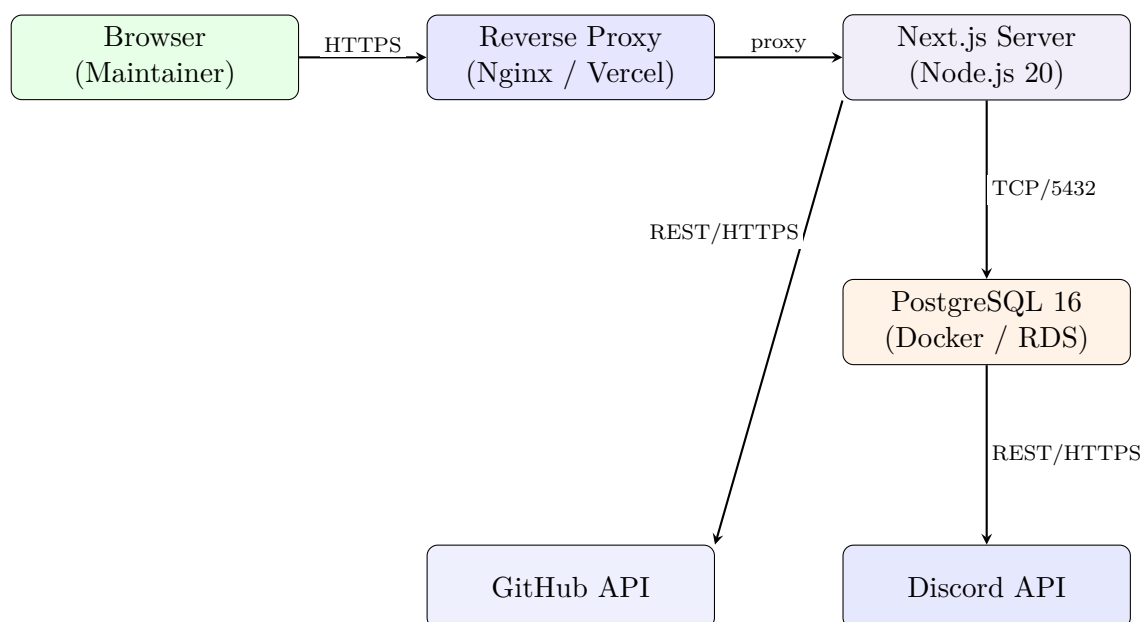


Figure 7.1: Production Deployment Architecture

7.2 Local Development Setup

Listing 7.1: Complete local bootstrap

```
1 # 1. Clone repository
2 git clone https://github.com/[org]/complaint-management-system
3 cd complaint-management-system
4
5 # 2. Install dependencies
6 npm install
7
8 # 3. Configure environment
9 cp .env.example .env
10 # Edit .env with your DATABASE_URL, GITHUB_TOKEN, DISCORD_BOT_TOKEN
11
```

```
12 # 4. One-command bootstrap (starts Docker, pushes schema, seeds data)
13 npm run setup
14
15 # 5. Start development server
16 npm run dev
17 # App available at http://localhost:3000
```

7.3 Environment Variables

Variable	Purpose
<code>DATABASE_URL</code>	PostgreSQL connection string.
<code>GITHUB_TOKEN</code>	GitHub Personal Access Token (repo scope).
<code>GITHUB_OWNER</code>	GitHub organisation or user owning the target repo.
<code>GITHUB_REPO</code>	Repository name to create issues in.
<code>DISCORD_BOT_TOKEN</code>	Discord Bot token for posting thread replies.

7.4 Screenshots of the Working Application

Instructions

Run the application locally with `npm run dev` after seeding demo data (`npm run db:seed`) and capture the following screenshots to insert below.

Screenshot 1 — Triage Inbox

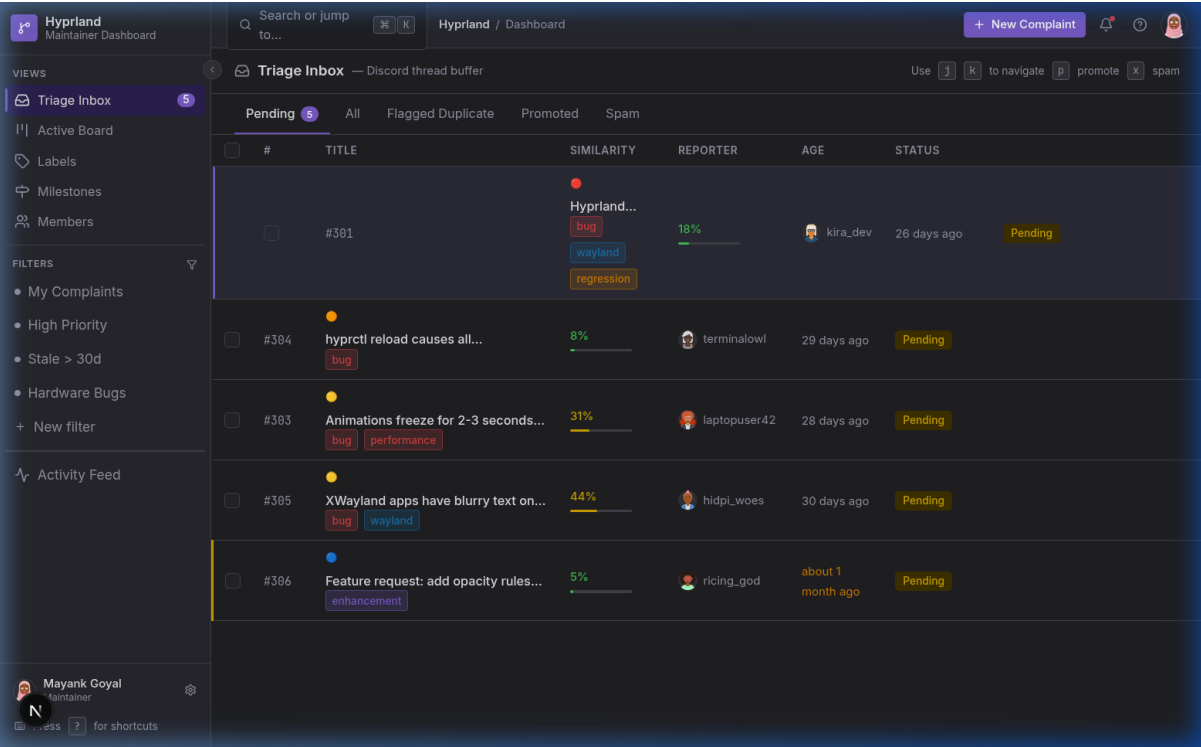


Figure 7.2: Triage Inbox — issue list with priority, source badges and action buttons

Screenshot 2 — Kanban Board

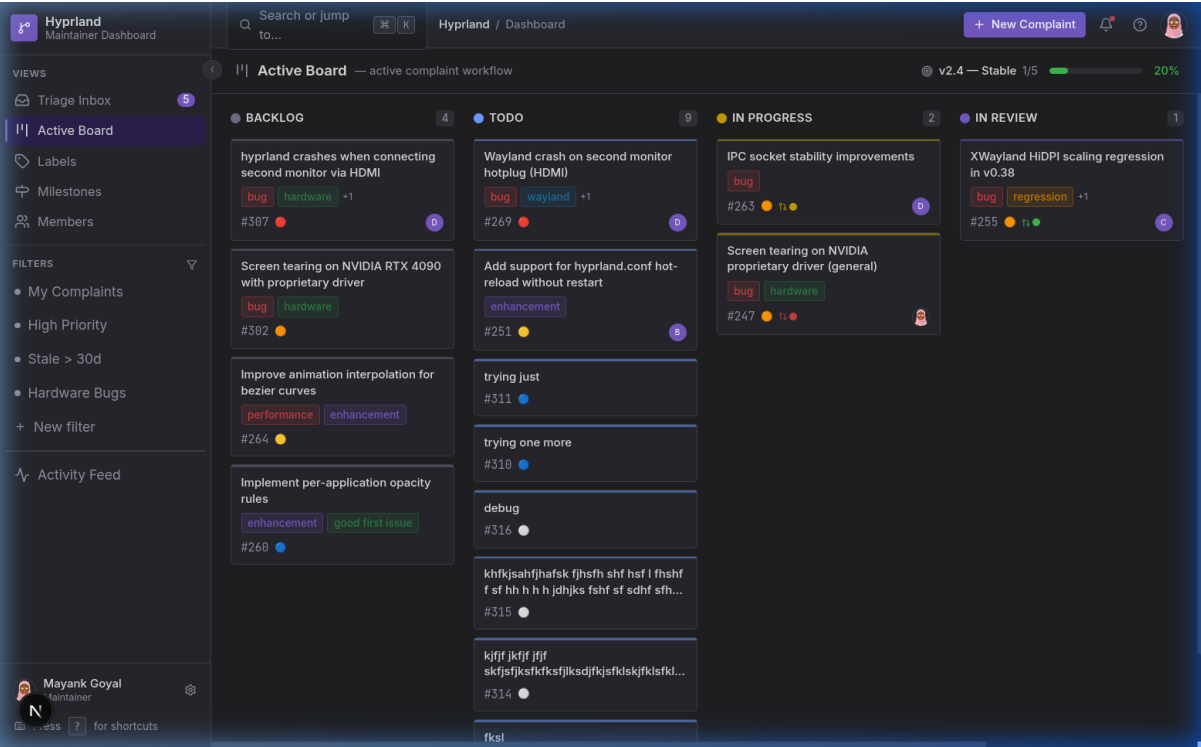


Figure 7.3: Kanban Board — five-column workflow view with drag-and-drop cards

Screenshot 3 — Issue Drawer

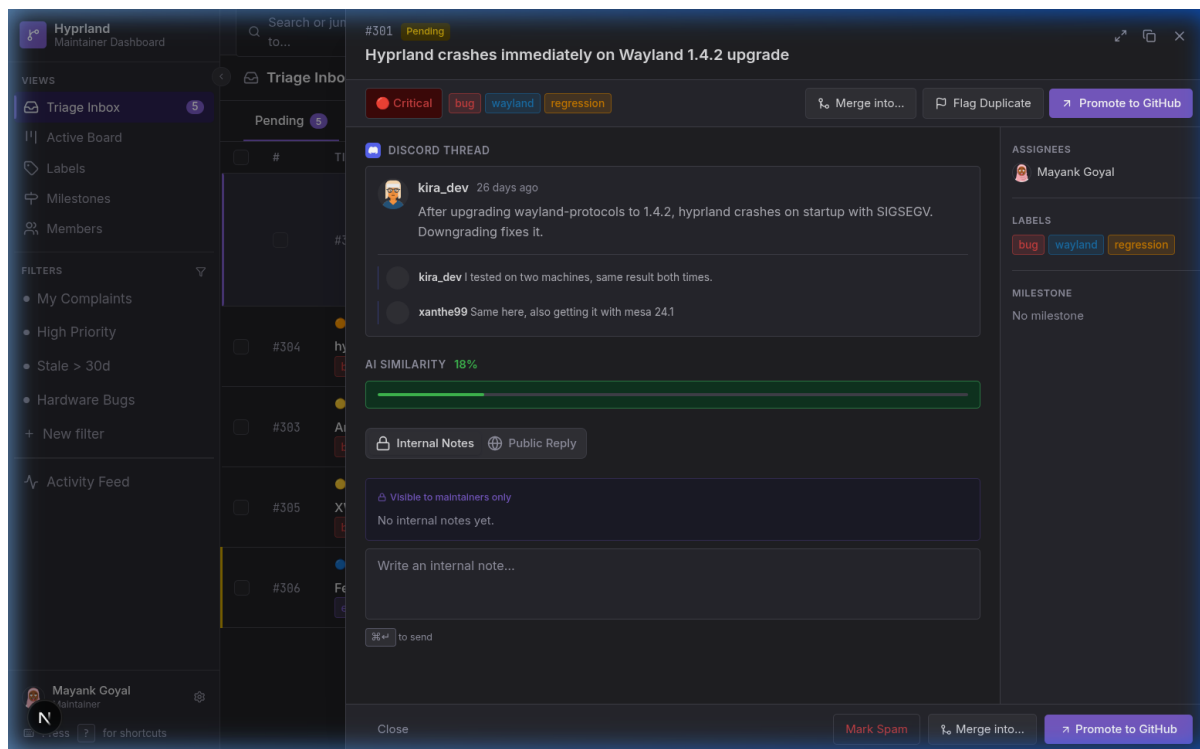


Figure 7.4: Issue Drawer — full issue detail with metadata sidebar and action panel

Screenshot 4 — Dashboard Overview

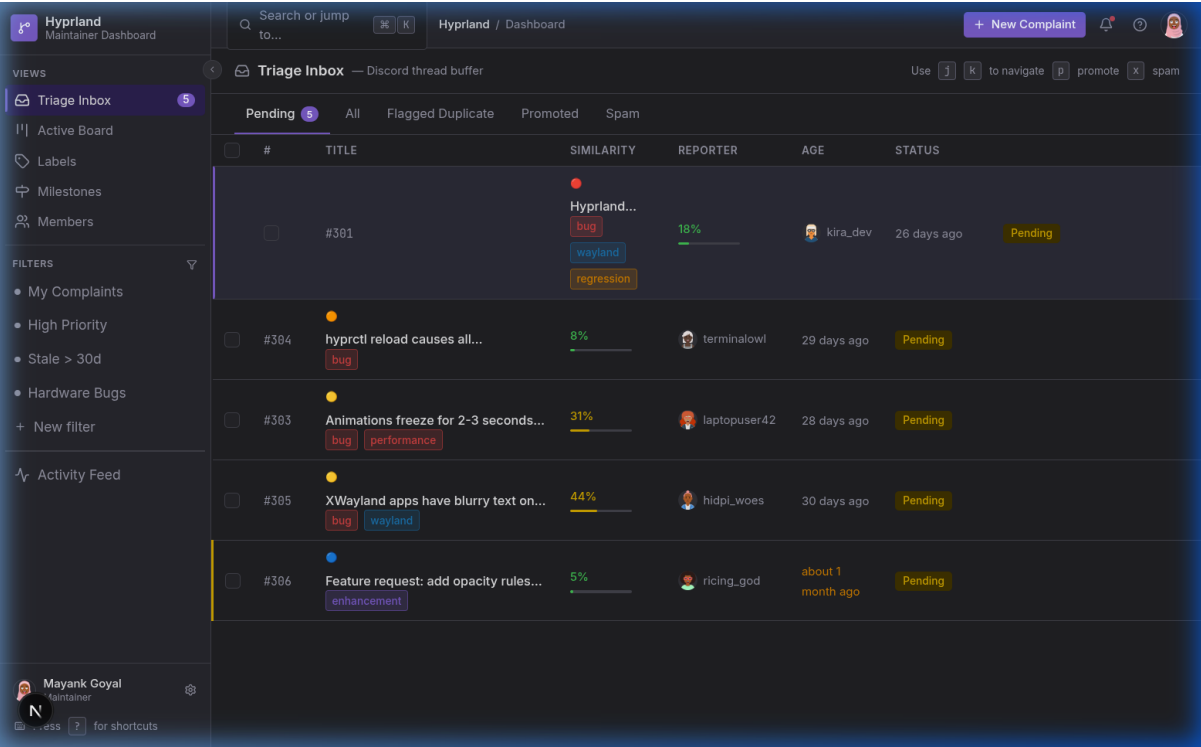


Figure 7.5: Dashboard Overview — navigation sidebar, triage count badge and main content area

Chapter 8

User Manual

8.1 Purpose and Scope

This manual explains how to use the Complaint Management System dashboard for day-to-day complaint triage and tracking. The app is designed for **maintainers** who need to:

- review incoming complaints from Discord,
- promote valid complaints into active work,
- track active work on a Kanban board,
- inspect full issue details in a side drawer,
- create new complaints directly.

Scope Clarification

The UI contains references to Discord and GitHub complaint sources, but direct external platform integrations require the relevant environment variables to be configured (`DISCORD_BOT_TOKEN`, `GITHUB_TOKEN`). When these are absent, the system degrades gracefully: promotion still works locally, but no external API calls are made.

8.2 Prerequisites

- Node.js 20 LTS and npm installed.
- Docker installed and running (for local PostgreSQL).

Start the application:

```
1 npm install
2 cp .env.example .env
3 npm run setup      # starts DB, pushes schema, seeds demo data
4 npm run dev        # starts Next.js dev server
```

Open `http://localhost:3000`. The root route redirects automatically to `/dashboard/triage`.

8.3 Dashboard Layout

The dashboard is divided into four persistent regions:

1. **Sidebar (left)** — Main navigation with links to Triage Inbox, Active Board, and Settings. A badge shows the count of pending triage items. The sidebar can be collapsed/expanded.
2. **Top Bar (top)** — Contains the organisation name, a Search / command palette trigger, and the “New Complaint” button.
3. **Main Content (centre)** — Renders the currently active page (Triage, Board, or Settings).
4. **Overlay Surfaces** — Issue Drawer (right panel), Create Complaint modal, Command Palette, and the global keyboard handler (invisible component).

8.4 Triage Inbox

Path: /dashboard/triage

8.4.1 Reading the Table

Each row in the triage table shows:

- **Checkbox** — for multi-select / bulk actions.
- **#** — sequential complaint number within the organisation.
- **Title** — complaint title with priority icon and label chips. A speech-bubble icon with count shows internal note activity.
- **Similarity** — AI-computed duplicate score rendered as a percentage and colour-coded bar:
 - **Green (< 30%)** — low similarity, likely unique.
 - **Amber (30–65%)** — possible duplicate, review recommended.
 - **Red (> 65%)** — high similarity, likely duplicate.
- **Reporter** — Discord avatar and username.
- **Age** — relative time since creation; turns amber if stale (> 30 days).
- **Status** — coloured badge (Pending, Spam, Promoted, ...).
- **Hover Actions** — Promote ([promote]), Spam ([spam]), Open ([open]) icons appear on row hover.

8.4.2 Filters

Use the tab bar at the top of the inbox to filter by: **Pending** | **All** | **Flagged Duplicate** | **Promoted** | **Spam**.

8.4.3 Keyboard-Driven Triage

Key	Action
j	Move cursor down one row.
k	Move cursor up one row.
Enter	Open Issue Drawer for the focused row.
p	Promote the focused complaint to GitHub.
x	Mark the focused complaint as spam.
e	Mark the focused complaint as done.
g t	Navigate to Triage Inbox.
g b	Navigate to Active Board.
g s	Navigate to Settings.
c	Open Create Complaint modal.
Ctrl+K	Open Command Palette.
Esc	Close open drawer or modal.
f	Toggle focus (wide) mode on drawer.

8.4.4 Bulk Actions

Select two or more complaints using their checkboxes. The Bulk Action Bar appears at the bottom of the screen with:

- **Promote All** — promotes each selected complaint sequentially.
- **Spam All** — marks all selected as spam in a single database query.
- **Clear Selection** — deselects all.

8.5 Active Board

Path: /dashboard/board

8.5.1 Columns

The board displays five workflow columns for GitHub-source issues:

1. **Backlog** — promoted but not yet scheduled.
2. **Todo** — queued for implementation.
3. **In Progress** — actively being worked on.
4. **In Review** — pull request open / awaiting review.
5. **Done** — completed.

8.5.2 Moving Cards

Drag a card from one column and drop it onto another. The UI updates optimistically and persists the change to the database. If the server call fails, the card animates back to its original position and an error toast appears.

8.5.3 Card Information

Each card shows:

- Issue number and truncated title.
- Priority dot (colour-coded).
- Label chips.
- Linked PR number with CI status dot.
- Assignee avatar stack.

Click any card to open the Issue Drawer for full context.

8.5.4 Milestone Progress

When a milestone is active, the board header shows the milestone title, a `closed/total` fraction, a progress bar, and a percentage.

8.6 Issue Drawer

8.6.1 Opening and Closing

- **Open:** click any triage row or board card; press `Enter` on the focused triage row.
- **Close:** press `Esc`; click the **X** button; click the backdrop; click the footer **Close** button.

8.6.2 Drawer Sections

Section	Content
Header	Issue number, status badge, stale indicator, title, focus/copy/close controls.
Quick Action Toolbar	Priority chip, label chips, and context-aware CTA buttons (Promote / Mark Spam for Discord; View on GitHub / Create Branch for GitHub-source).
Discord Thread Block	(Discord source only) Original message and replies in a threaded layout with Discord-brand colours.
Issue Description	(GitHub source) Plain-text description block.

Section	Content
AI Similarity Block	Coloured progress bar with score; if <code>duplicateOfId</code> is set, shows “Merge into #N” and “Not a duplicate” buttons.
Dual Chat Tabs	Internal Notes tab (purple border, lock icon, maintainer-only) and Public Reply tab (green border, globe icon, will post to Discord).
Metadata Sidebar	Assignees, Labels, Milestone, GitHub issue link, Linked PRs with CI dots.
Footer	Close, Mark Spam, Merge into (phase-2), Promote to GitHub buttons.

8.7 Create Complaint

Open the modal by pressing **c** or clicking **New Complaint** in the top bar.

Field	Details
Title	Required. Maximum 200 words.
Description	Optional. Markdown-friendly plain text area.
Priority	Drop-down: No Priority, Low, Medium, High, Critical.

Submit with **Create Complaint** button or **Ctrl+Enter** (**Cmd+Enter** on macOS). The new complaint appears in the Active Board’s **Todo** column.

8.8 Command Palette

Open with **Ctrl+K** (or **Cmd+K** on macOS).

The palette contains three groups:

- **Navigate** — Go to Triage, Board, Settings.
- **Actions** — Promote, Mark Spam, Mark Done, Assign, Merge, Flag Duplicate.
- **View** — Toggle Focus Mode.

Type to fuzzy-search commands. Press **Enter** to execute the highlighted command.

8.9 Troubleshooting

Symptom	Resolution
App does not load / DB error	Ensure Docker is running. Run <code>npm run db:up && npm run db:push</code> .
Empty triage or board	Run <code>npm run db:seed</code> to insert demo data.

Symptom	Resolution
Port 5432 conflict	Stop the conflicting local PostgreSQL service or edit <code>DATABASE_URL</code> in <code>.env</code> and update <code>scripts/db-up.sh</code> .
Stale UI after action	Hard-refresh the browser (<code>Ctrl+Shift+R</code>). Check the terminal for Prisma/runtime errors.
GitHub promotion fails	Verify <code>GITHUB_TOKEN</code> , <code>GITHUB_OWNER</code> , <code>GITHUB_REPO</code> in <code>.env</code> are correct and token has <code>repo</code> scope.
Discord reply not sent	Verify <code>DISCORD_BOT_TOKEN</code> is set and the bot has Send Messages permission in the target forum channel.

8.10 Daily Workflow Recommendation

1. Start in the **Triage Inbox** (Pending filter).
2. Use `j/k` to navigate; press `Enter` to open the drawer for any complaint that warrants detailed review.
3. Press `p` to promote valid complaints; `x` to spam obvious noise.
4. After triage, switch to the **Active Board** (`g b`).
5. Drag cards across columns to reflect actual work state.
6. Use the **Internal Notes** tab in the drawer for team communication about sensitive fixes.
7. Use **Public Reply** tab to send visible updates back to the original Discord reporter.

Chapter 9

Summary

9.1 Project Achievements

The Complaint Management System successfully delivers a **specialised, keyboard-first triage ecosystem** that addresses the real-world pain-points of open-source maintainers. The following capabilities were implemented and are fully functional in the submitted build:

Feature	Status
Triage Inbox	Implemented — table, filters, cursor, keyboard actions.
Promote Pipeline	Implemented — GitHub API, Discord reply, DB update.
Spam / Done Actions	Implemented — persist to DB, revalidate.
Bulk Promote / Spam	Implemented — multi-select, batch DB update.
Kanban Board	Implemented — DnD, optimistic update, 5 columns.
Issue Drawer	Implemented — animated, fetch, full context.
AI Similarity Display	Implemented — score bar, colour coding, duplicate hint.
Create Complaint	Implemented — modal, validation, DB persist.
Command Palette	Implemented — cmdk, grouped commands.
Keyboard Shortcuts	Implemented — tinykeys, j/k/p/x/e/f/g-seq.
Discord Chat Integration	Phase 2 — UI renders thread; live bot requires token.
Note Persistence	Phase 2 — UI shows toast; actual save deferred.
Merge / Flag Duplicate	Phase 2 — buttons present; action shows toast.
Assign to Me	Phase 2 — keyboard event fires; DB write deferred.
Public Read-Only Mirror	Phase 2 — architecture designed, not deployed.

9.2 Key Design Decisions

1. **Server Actions over Traditional API Routes** — Using Next.js Server Actions for all mutations eliminates the need for a separate REST API layer for writes, keeping secrets server-side automatically and reducing boilerplate significantly.
2. **Zustand for Local UI State** — The drawer open/close state, bulk selection set, and triage cursor index are lightweight client-only concerns. Zustand provides these without the overhead of a global server state manager.
3. **Prisma Schema-as-Truth** — Using `prisma db push` in development means the schema file is the single source of truth for the database structure, eliminating schema drift.
4. **Optimistic UI for Kanban** — The board update applies immediately on drag-end, making the experience feel instantaneous. Rollback on failure ensures consistency.
5. **Graceful Integration Degradation** — Both GitHub and Discord integrations are wrapped in feature-flag checks (`isGitHubIntegrationEnabled()`, `isDiscordBotEnabled()`). This means the application is fully functional as a standalone tool without any external accounts.

9.3 Lessons Learned

- React Server Components require careful boundary placement: data fetching belongs in RSCs, event handlers and state belong in Client Components marked "use client".
- The `@dnd-kit` library required careful configuration of collision detection (`closestCorners`) to correctly identify the target column when dragging between columns of different sizes.
- Discord's `discordReplies` JSON field (stored as `Json?`) provides flexibility but requires defensive null-checking throughout the UI layer.
- The `revalidatePath` calls after Server Actions are critical for keeping server-rendered pages in sync after mutations; missing a call results in stale data being displayed.

9.4 Future Work

1. Implement full **NextAuth.js** authentication with GitHub OAuth and organisation membership verification.
2. Build the **AI embedding pipeline** (e.g., using `pgvector` + OpenAI embeddings) to compute and store `similarityScore` in real time on issue ingest.
3. Deploy the **public read-only mirror** as a statically generated Next.js site with ISR (Incremental Static Regeneration).

4. Implement **note persistence** (complete the `createNote` server action wired to the textarea submit).
5. Add **label and assignee management** UI in the Settings page.
6. Implement the **merge-duplicate** workflow in full, including status update, cross-linking, and Discord notification.
7. Add **webhook ingestion** endpoint to receive Discord Forum events automatically rather than requiring manual data import.